

Table of Contents

Week 1 — What is OOP?

History, Benefits & OOP vs Procedural Programming

Lesson Overview

Hour Plan

Part 1 — What Is a Programming Paradigm? (10 min)

Key Idea

Major Paradigms

Discussion Prompt (2 min)

Part 2 — Problems with Procedural Programming (10 min)

What is Procedural Programming?

A Procedural Example (Bank System)

Why Procedural Breaks Down at Scale

One-Line Summary

Part 3 — How OOP Was Born (10 min)

Brief History Timeline

Alan Kay's Original Vision (share with students)

Why OOP Became Dominant

Part 4 — The 4 Pillars of OOP (10 min)

Overview Diagram

Real-World Analogy: A University

Part 5 — Hands-On: First Java Class (15 min)

Step-by-Step IDE Instructions

OOP vs Procedural — Side-by-Side Comparison

Test Checkpoint

Common Misconceptions

Extension Challenges



Basic



Intermediate



Advanced

Lesson Summary

Homework

Week 1 — What is OOP?

History, Benefits & OOP vs Procedural Programming

Date: 27 May – 2 June | **Duration:** 1 Hour | **Course:** Object-Oriented Programming Concepts

Lesson Overview

Item	Detail
Topic	Introduction to Object-Oriented Programming
Duration	1 hour
Format	Lecture + Discussion + Live demo
Prerequisites	None — this is the opening lesson
Tools	Whiteboard / slides, IDE (IntelliJ IDEA or VS Code + Java extension)
Outcome	Students can explain OOP in plain English, name its 4 pillars, and run their first Java class

Hour Plan

00:00 – 00:10	Part 1 – What is a programming paradigm?
00:10 – 00:20	Part 2 – Problems with procedural programming
00:20 – 00:30	Part 3 – How OOP was born (brief history)
00:30 – 00:40	Part 4 – The 4 Pillars of OOP (overview only)
00:40 – 00:55	Part 5 – Hands-on: First Java class, run it together
00:55 – 01:00	Recap, checkpoint, homework

Part 1 — What Is a Programming Paradigm? (10 min)

Key Idea

A **programming paradigm** is a style or way of thinking about and structuring a program. It is not a language — it is an approach. Different paradigms give different answers to the question: *"How should I organize my code?"*

Major Paradigms

PROGRAMMING PARADIGMS

- Procedural (Structured)
"Organize code as a sequence of instructions and functions"
Languages: C, Pascal, BASIC
- Object-Oriented (OOP)
"Organize code as interacting objects, each with data and behavior"
Languages: Java, Python, C++, C#, Kotlin
- Functional
"Organize code as mathematical functions – avoid changing state"
Languages: Haskell, Erlang, (Scala, Kotlin partially)
- Declarative
"Describe WHAT you want, not HOW to do it"
Languages: SQL, HTML

Discussion Prompt (2 min)

Ask students: *"When you cook a recipe, you follow step-by-step instructions. That is like procedural programming. But when you think of a restaurant kitchen — it has a Head Chef, Sous Chef, Pastry Chef, each with their own tools and responsibilities — that is more like OOP. Which approach do you think works better as a system grows?"*

Part 2 — Problems with Procedural Programming (10 min)

What is Procedural Programming?

In procedural programming, a program is a sequence of **functions** operating on **data**. Functions and data are separate — there is no natural bond between them.

A Procedural Example (Bank System)

```
// PROCEDURAL STYLE – data and functions are disconnected
// All data lives as loose variables or arrays

String accountOwner   = "Sokha";
double accountBalance = 1000.0;
String accountType    = "Savings";

// Functions are separate from data
static void deposit(double balance, double amount) {
    balance += amount;
    System.out.println("New balance: " + balance);
}

static void withdraw(double balance, double amount) {
    if (amount > balance) {
        System.out.println("Insufficient funds");
        return;
    }
    balance -= amount;
}

static void printStatement(String owner, double balance) {
    System.out.println(owner + ": $" + balance);
}
```

Why Procedural Breaks Down at Scale

PROBLEM 1 – Data is unprotected

Anyone can write: `accountBalance = -999999`;
There is no rule preventing invalid values.

PROBLEM 2 – Functions and data are not connected

`deposit()` knows nothing about `accountOwner`.
`printStatement()` knows nothing about `accountType`.
You must pass everything as parameters manually.

PROBLEM 3 – Difficult to model the real world

A bank has accounts, customers, transactions, loans.
Modeling all of this as loose variables and functions
becomes unmanageable quickly.

PROBLEM 4 – Hard to reuse and extend

To add a "Premium Account", you copy-paste and modify.
Every change risks breaking existing functions.

PROBLEM 5 – No clear ownership

Which function is responsible for which data?

In a large team, this causes conflicts and bugs.

One-Line Summary

*In procedural programming, **data and behavior live separately** — and this separation becomes a serious problem as programs grow.*

Part 3 — How OOP Was Born (10 min)

Brief History Timeline

1960s – Simula (Norway)

Created by Ole-Johan Dahl and Kristen Nygaard at the Norwegian Computing Center. Designed to simulate real-world systems.

First language to introduce the concept of "class" and "object".

1970s – Smalltalk (Xerox PARC)

Alan Kay coined the term "Object-Oriented Programming".

Everything in Smalltalk is an object – including numbers and booleans.

Alan Kay's vision: objects sending messages to each other, like independent mini-computers.

1980s – C++ (Bell Labs)

Bjarne Stroustrup added OOP features to the C language.

Made OOP practical for systems programming and large-scale software.

1995 – Java (Sun Microsystems)

"Write once, run anywhere."

Java was designed from the ground up as an OOP language, bringing OOP to the mainstream of enterprise software development.

2000s–Today

Python, C#, Kotlin, Swift – OOP is the dominant paradigm for application development worldwide.

Alan Kay's Original Vision (share with students)

"The big idea is messaging... The key in making great and growable systems is much more to design how its modules communicate rather than what their internal properties and behaviors should be."
— Alan Kay

Why OOP Became Dominant

OOP maps naturally to **how humans think about the world**. We do not think of the world as a series of procedures — we think of it as *things* that have *properties* and can *do actions*.

HUMAN THINKING	→	OOP CONCEPT
"I have a car"	→	Object (instance)
"Cars have a color"	→	Attribute (field)
"My car is red"	→	Object state
"I can drive the car"	→	Method (behavior)
"All cars work this way"	→	Class (blueprint)

Part 4 — The 4 Pillars of OOP (10 min)

Note to instructor: This section is an overview only. Each pillar will be taught in full detail in later weeks. The goal here is to give students a mental map of where the course is going.

Overview Diagram

OOP – 4 PILLARS	
ENCAPSULATION (Week 4)	Bundle data + behavior together Hide internal details → ATM machine: you see buttons, not the vault mechanism
INHERITANCE (Weeks 6–7)	Child class acquires parent's properties and behaviors → Student IS-A Person
POLYMORPHISM (Week 9)	Same operation, different forms depending on the actual object

	→ "speak()" on Dog vs Cat
ABSTRACTION (Weeks 10–11)	Expose only what is necessary Hide unnecessary complexity → Car has accelerate() – you don't see the engine internals

Real-World Analogy: A University

UNIVERSITY (as an OOP system)

- Student object
 - Attributes: name, studentId, gpa
 - Behaviors: enroll(), submitAssignment(), checkGrade()
- Teacher object
 - Attributes: name, subject, yearsOfExperience
 - Behaviors: teach(), gradeExam(), holdOfficeHours()
- Course object
 - Attributes: courseName, credits, maxEnrollment
 - Behaviors: addStudent(), removeStudent(), getEnrolledCount()
- Department object
 - Attributes: deptName, deptCode
 - Behaviors: addCourse(), listTeachers(), generateReport()

Each object:

- Bundles its own data and behavior (Encapsulation)
- May be built on top of a common base (Inheritance)
- Can be treated as its general type (Polymorphism)
- Exposes only what is needed (Abstraction)

Part 5 — Hands-On: First Java Class (15 min)

Step-by-Step IDE Instructions

Step 1 — Open your IDE (IntelliJ IDEA or VS Code)

Step 2 — Create a new Java project

- IntelliJ: `File → New → Project → Java → Name: "OOP_Week1"`

- VS Code: Create folder `OOP_Week1`, open terminal, run `javac --version` to confirm Java is ready

Step 3 — Create a new file named `Student.java`

- Right-click `src` folder → `New → Java Class` → name it `Student`

Step 4 — Type the following class exactly as shown

```
// Student.java
// This is a CLASS – a blueprint describing what every Student object will have

public class Student {

    // ATTRIBUTES – the data that every Student object holds
    // (We will make these private in Week 4 – for now, keep them simple)
    String name;
    int    age;
    String major;

    // BEHAVIORS – the actions a Student object can perform
    void introduce() {
        System.out.println("Hi! My name is " + name);
        System.out.println("I am " + age + " years old");
        System.out.println("I study " + major);
        System.out.println("---");
    }
}
```

Step 5 — Create a new file named `Main.java`

```
// Main.java
// This is where we CREATE objects from the Student blueprint and USE them

public class Main {

    public static void main(String[] args) {

        // Create Object 1 – a specific Student
        Student s1 = new Student();    // 'new' builds an actual object in memory
        s1.name  = "Sokha";
        s1.age   = 20;
        s1.major = "Computer Science";

        // Create Object 2 – a different Student, completely independent
        Student s2 = new Student();
        s2.name  = "Dara";
        s2.age   = 22;
        s2.major = "Engineering";

        // Ask each object to introduce itself
        s1.introduce();
        s2.introduce();
    }
}
```



```
// Key observation: both objects use the SAME class (blueprint)
// but each has its own independent data
System.out.println("Both students came from ONE blueprint: the Student class");
} }
```

Step 6 — Run `Main.java`

- IntelliJ: Right-click `Main.java` → `Run Main.main()`
- VS Code terminal: `javac Main.java Student.java && java Main`

Step 7 — Confirm the expected output:

```
Hi! My name is Sokha
I am 20 years old
I study Computer Science
---
Hi! My name is Dara
I am 22 years old
I study Engineering
---
Both students came from ONE blueprint: the Student class
```

Step 8 — Live experiment: change `s1.name = "Sokha"` to `s1.name = "Your own name"` and re-run. Ask students: did `s2` change? (No — they are independent objects.)

OOP vs Procedural — Side-by-Side Comparison

SOLVING THE SAME PROBLEM – Track two students' information

PROCEDURAL APPROACH

```
String name1 = "Sokha";
int    age1  = 20;
String major1 = "CS";

String name2 = "Dara";
int    age2  = 22;
String major2 = "Engineering";

printStudent(name1,age1,major1);
printStudent(name2,age2,major2);
```

OOP APPROACH

```
Student s1 = new Student();
s1.name = "Sokha";
s1.age  = 20;
s1.major = "CS";

Student s2 = new Student();
s2.name = "Dara";
s2.age  = 22;
s2.major = "Engineering";

s1.introduce();
s2.introduce();
```

```
String name3 = ...  
int age3 = ...  
String major3 = ...  
printStudent(name3, age3, major3)
```

```
Student s3 = new Student();  
s3.name = ...  
(same pattern - clean)
```

- ✗ Data scattered everywhere
- ✗ No structure or ownership
- ✗ Functions and data separate
- ✗ Hard to scale

- ✓ Data grouped inside object
- ✓ Object owns its own data
- ✓ Methods travel with data
- ✓ Easy to add more objects

Test Checkpoint

By the end of this lesson, each student should be able to:

- ☐ Explain the difference between OOP and procedural programming in one sentence
- ☐ Name the 4 pillars of OOP and give a one-line description of each
- ☐ Run `Main.java` and see the correct output for two student objects
- ☐ Explain why `s1.name` and `s2.name` are independent (in their own words)

Common Misconceptions

Misconception	Clarification
"OOP is just Java syntax"	OOP is a thinking approach. Java is one language that supports it. Python, C++, and others also support OOP.
"Procedural is bad, OOP is good"	Both paradigms have valid uses. OOP shines for large, complex systems. Procedural can be simpler for small scripts.
"I need to understand all 4 pillars today"	Not at all. Today is just the map. The journey starts next week.
"A class and an object are the same thing"	No. A class is a blueprint. An object is a real instance created from it. A city has one blueprint for a type of house, but hundreds of actual houses built from it.
"OOP means everything must be a class"	OOP is a paradigm — not every line must be in a class, but the primary organization unit is the class.

Extension Challenges

Basic

Open a notebook (physical or digital) and pick any 3 real-world things around you (phone, bag, chair).

For each, write:

- What **attributes** does it have? (color, weight, brand, etc.)
- What **behaviors** can it perform? (ring, vibrate, carry items, etc.)
- Would it make sense as a Java class? Why?

Intermediate

Without using any code — only a diagram or written description — design an OOP model for a **Hospital System**.

Include at least: `Patient`, `Doctor`, `Appointment`, `MedicalRecord`.

For each, list 3 attributes and 2 behaviors. Draw lines to show which objects interact with which.

Advanced

Read the following statement and write a response (1–2 paragraphs):

> “Alan Kay, the inventor of OOP, later said he regretted calling it ‘object-oriented’ and felt the word ‘object’ drew too much attention. He said the key idea was really messaging — objects communicating with each other.”

Do you agree? Based on what you learned today, explain what “messaging” means in the context of OOP and whether the name “Object-Oriented” captures or misses the real idea.

Lesson Summary

TODAY'S KEY TAKEAWAYS

1. A programming paradigm is a way of organizing code.
2. Procedural programming = functions + data, separate.
OOP = objects that bundle their own data AND behavior.
3. OOP maps naturally to how humans understand the world:
things (objects) that have properties (attributes)
and can do things (methods).
4. OOP was invented to solve the growing complexity of
large software systems.

5. The 4 Pillars of OOP:

- Encapsulation (hide and protect data)
- Inheritance (child reuses parent)
- Polymorphism (same interface, different behavior)
- Abstraction (expose only what is needed)

6. A CLASS is a blueprint.

An OBJECT is an instance built from that blueprint.

Homework

Create a new file `Animal.java` and a `MainAnimal.java`.

Design a `Animal` class with:

- **Attributes:** `name`, `species`, `age`
- **Behaviors:** `eat()` (prints "name is eating"), `sleep()` (prints "name is sleeping")

In `MainAnimal.java`:

- Create **3 different** `Animal` objects with different data
- Call both methods on each animal
- Verify that changing one animal's name does not affect the others

Bring your code next week — we will use `Animal` as a starting point for Week 2.

Week 1 of 16 | Object-Oriented Programming Concepts | @KhmerSide | syuthd.com